

Programming for Analytics

Week 3: Flow Control · Pseudo-code · UML Activity Diagrams

*Day 3 at The Daily Grind: the register finally gets to **decide**, and we learn to draw programs before we write them.*

Romain Claret · University College Dublin, 2026

Where you are

Week 3 of 10

You can already: variables · types · f-strings · `input()` · files · `argv` · booleans · functions · scope · modules

Today you add: `if/elif/else` · `while` · `for` · `range` · iterables · **UML activity diagrams** · pseudo-code · versions & testing discipline

Still ahead: lists/dicts/sets/tuples (W4) · memory & unbreakable code (W5) · numpy + pandas + **your project** (W6, a double week) · classes (W7) · functional + GUIs (W8) · complexity (W9) · testing & exam prep (W10)

How last week landed

Self-check 2 closed last night.

Friday's close poll, 35 of you: today's pace



- too slow `0%`
- a little slow `6%`
- just right `26%`
- a little fast `40%`
- too fast `28%`

All 132 of you: 97 did not answer



- did not answer `97`
- a little slow `2`
- just right `9`
- a little fast `14`
- too fast `10`

Only 35 of 132 answered: too few to speak for the room. Answer today's poll, and it can.

Your close poll: asked above everything else: *reading and writing files, and splitting a program across two files*. The rest of your questions are answered on the **Q&A** page, under Feedback on Brightspace.

Self-check 2, the hardest question (63% right first time):

```
first = log.___() # exactly one line from the file
```

§0 · Outline: what's in this file

§	Section	Café / concept	Page
1	Think first	flowcharts → pseudo-code → UML activity diagrams	7
2	Decisions (`if` / `elif` / `else`)	the discount finally ACTS; dangling-else	11
3	Repetition (`while`)	the muffin rush; the menu loop; `take_order`	17
	Break		23
4	Element-wise (`for` & iterables)	argv, strings, `in`, files, `range()`; day summary	24
5	How professionals build	versions, testing culture, docstrings	32
6	Classic traps	the three that catch everyone	35
7	Exam & exercises · checklist		37

This week's café feature: the register decides (discount!), serves customers **until we close** (menu loop), and reads back the day (a `for` over the receipts). **New concepts:** pseudo-code; **UML activity diagrams**; `if`/`elif`/`else`; `while`; `for` + iterables + membership `in` + `range()`.

Where our café is, and the itch we left

The Week-2 café is a **two-file program** (`cafe_w2.py` + `register_sayings`): it takes a fixed-form order, prints an **aligned receipt** (`9.60`, we fixed that), and its revenue **survives closing time**. But two things nagged us:

```
Big order (over 10.00)?  True        ← computed... and then NOTHING happened
How many espressos? How many lattes? ← one hard-coded question per product, one order per run
```

The register could **answer** questions but not **act** on them, and it served **one** customer per run. Both walls fall today. And we adopt the habit that separates programmers from typists: **we draw the program before we write it.**

You asked: `open` hands you a bookmark, not the text

`open()` reads nothing. It hands back a **file object**: where the file is, plus a **bookmark** for how far into it you are. Every read moves that bookmark forward.

menu.txt	bookmark	
`espresso, single shot 2.50`	0	← `open("menu.txt")`: nothing read yet
`latte, with oat milk 3.20`	27	← after one `readline()`
`blueberry muffin, warm 2.40`	53	← after two
`tea, pot for one 2.20`	81	← after three
	103	← `read()` jumps to the end; the next gives `''`

- **`readline()` leaves the bookmark on the next line,** which is why two calls give two different lines. `read()` takes the rest of the file in one go, so it was the wrong answer on self-check 2.
- **`tell()` reads that number, `seek(n)` moves it.** The right-hand column is all they are.
- **Writing uses the same bookmark, the other way.** `"w"` empties the file, `"a"` adds to the end, `write()` takes strings only, and nothing is on disk until `close()`.

For the exam: **`open` gives a bookmark · `readline` a line · `read` to the end · `close` puts it down.**

§1 · CONCEPT Business analytics *is* decisions

What can our programs do so far? **Receive** data (keyboard, files, arguments), **process** it (calculations, comparisons, functions), **output** it (screen, files).

What's missing: *doing different things depending on the data.* Every business rule is a decision: *if the order is big, discount it; while stock remains, sell.*

We need flow control. And because flow is about *paths*, not lines of text, the right first tool isn't code; it's a **drawing**. You've met flowcharts before: assembly instructions, "is the lamp plugged in?" troubleshooting posters. Today we make them precise.

§1 · CONCEPT Pseudo-code: the rough idea in plain words

Before syntax, write the logic as a numbered mix of English and code. Example: *tell the user whether a number is even or odd*:

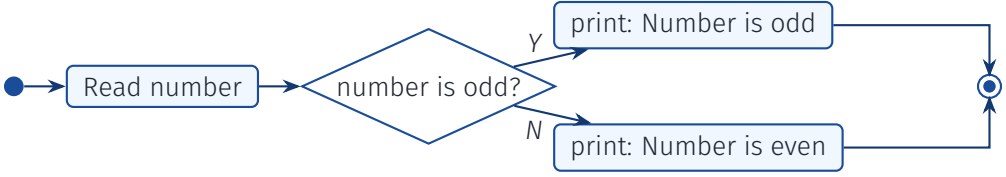
1. Ask the user for a number;
2. Compute the remainder of dividing it by 2 *(Week 1's %!)*;
   - if it is 1, print "The number is odd.";
   - if it is 0, print "The number is even.";
3. Quit.

That's **pseudo-code**: a rough idea of the program, written for humans. No interpreter will ever see it; it exists so *you* can think.

The UML activity diagram: the official notation

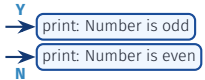
The same logic, drawn. **UML** (Unified Modeling Language) activity diagrams have exact conventions, and the exam uses them:

- ● **a filled black dot:** the *start* of the flow;
- **round-cornered rectangles:** *actions*;
- ◇ **diamonds:** *decision nodes* (label the outgoing edges!);
- ◉ **a dot with an extra circle:** the *end*;
- **directed edges (arrows):** the flow of control.



§1 · CONCEPT The triptych: diagram ↔ English ↔ code

One idea, three faithful representations. Learn to translate **in both directions** (the exam can ask for either):

The diagram says	The English says	The Python says
	1. Read a number	`number = input("Number? ")`
	2. If odd, say odd;	`if int(number) % 2 == 1:`
	…print “odd” else print “even”	`    print("Number is odd.")` `else:    print("Number is even.")`
	3. Quit	*(end of program)*

Same program, three notations. An exam question can come in from any of the three and ask for another, so practice the translation, not the picture.

§2 · CAFÉ `if`, and the cliffhanger falls

The syntax, and the payoff, in one breath:

```
if boolean_condition:
    command          # runs only when the condition is True
    command
command              # back to unindented = always runs
```

Last week the register computed `big_order = total > 10.0`, and could only *print* it. **Now:**

```
if big_order:
    total = total * 0.90     # 10% off: the boolean finally STEERS the program
```

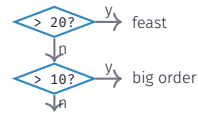
The indented block runs **only** when the condition is `True`. That one keyword turns our calculator-with-opinions into software that *acts*.

§2 · CONCEPT `if` / `elif` / `else`: the full family

Grading an order's size, all three forms:

```
if total > 20.0:
    print("A feast!")         # if: only when True
elif total > 10.0:
    print("A big order.")     # elif: else-if, checked in order, first hit wins
else:
    print("A quick coffee.") # else: the catch-all, when nothing above matched
```

- Conditions are checked **top to bottom**; the **first** `True` block runs; the rest are skipped.
- `else` never has a condition. `elif` can repeat as often as you need.
- These are exactly the **diamonds** of an activity diagram, stacked.



§2 · CAFÉ The discount, built properly (the 5 beats, fast)

① **Ask:** 10% off orders over €10. ② **Decompose:** one decision. ③ **Design:** a *function* (Week 2!) that returns the adjusted total, with named constants, not magic numbers. ④ **Build** (this is the file, verbatim):

```
DISCOUNT_THRESHOLD = 10.0      # orders over this get a discount
DISCOUNT_RATE = 0.10           # 10% off

def apply_discount(total):
    """Happy-hour rule: a percentage off orders over the threshold."""
    if total > DISCOUNT_THRESHOLD:
        return total * (1.0 - DISCOUNT_RATE)
    else:
        return total
```

⑤ **Test:** `9.99` → unchanged · `10.00` → unchanged (`>` not `>=`, a *decision we made*) · `20.00` → `18.00`. And the **docstring:** every function from here on gets one.

Nested `if`s

An indented block can contain *another* `if`. Decisions inside decisions:

```
if hard:
    if glitters:
        print("gold")
    else:
        print("useless")
```

Read it with the indentation: *if hard and glitters → gold; if hard but not glittering → useless; if not hard → nothing at all.*

Fine. But now a famous puzzle from language design…

The dangling else

In many languages this line really is ambiguous:

if hard: if glitters: gold else useless

Does the `else` belong to the **first** `if` or the **second**? Two readings, two different programs. **Python kills the ambiguity with indentation:**

```
if hard:                          |  if hard:
    if glitters:                  |      if glitters:
        print("gold")             |          print("gold")
    else:                         |  else:
        print("useless")          |      print("useless")
# else pairs with GLITTERS        |  # else pairs with HARD
```

Left: not-glittering → "useless". Right: not-hard → "useless". **The indentation *is* the meaning.** (A classic trap. Nested `if`s come back in Week 9, when we count the branches a program carries.)

§2 · TRY IT Predict, then run (3 min)

With `total = 12.0` and `items = 2`, what does each print?

```
if total > 10.0:
    print("A")
if total > 20.0:
    print("B")
else:
    print("C")

if total > 10.0:
    if items > 3:
        print("D")
else:
    print("E")
```

(Answers: A, C, and the third block prints nothing at all. If you said E, look at the indentation again: that `else` belongs to the outer `if`, which was True.)

§3 · CAFÉ The muffin rush: why we need `while`

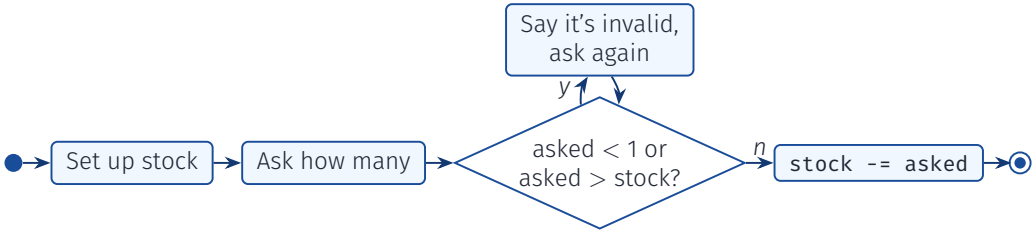
Saturday. We have **15 muffins**. A customer wants “some”. They can’t buy more than we have (or fewer than one), so we keep asking until the number makes sense. With our tools:

```
stock = 15
asked = int(input("How many muffins? "))
if asked < 1 or asked > stock:
    print(f"You can't buy {asked} muffins.")
    asked = int(input("How many muffins? "))
    if asked < 1 or asked > stock:
        print(f"You can't buy {asked} muffins.")
        asked = int(input("How many muffins? "))
        if asked < 1 or asked > stock:
            ...
```

Horrible code, and it only survives a few wrong answers. A stubborn customer defeats our program. We need *“keep asking WHILE the answer is invalid”*.

Muffin rush: pseudo-code → diagram → `while`

1. Set up the stock; 2. Ask how many; 3. **While** the answer is invalid, keep asking; 4. Update the stock.




```
stock = 15
asked = int(input("How many muffins? "))
while asked < 1 or asked > stock:
    print(f"You can't buy {asked} muffins.")
    asked = int(input("How many muffins? "))
stock -= asked
```

A nest with no end became **three plain lines**. The loop-back edge in the diagram *is* the `while`.

§3 · CONCEPT `while`: the syntax and the sentinel

```
while boolean_condition:
    command        # repeats as long as the condition is True
    command
command            # runs once the condition turns False
```

The other shape worth memorizing: repeat **until a signal**:

```
nbr = int(input("A number and I'll square it (0 to quit): "))
while nbr != 0:
    print(f"The square is {nbr * nbr}.")
    nbr = int(input("Another one (0 to quit): "))
print("The end.")
```

`0` is the **sentinel:** the value that means “stop”. Something in the body must be able to change the condition, or the loop never ends.

The menu loop: the shape you'll reuse all term

Every register, ticket machine and ATM runs a **menu loop**, and so will your **project's trading desk**:

```
choice = ""
while choice != "0":
    print("\n[1] New order   [2] Today's revenue   [3] Day summary   [0] Close up")
    choice = input("Choose: ").strip()
    if choice == "1":
        ...          # then elif for 2, 3 and 0: one branch each
    else:
        print("Please choose 1, 2, 3 or 0.")   # junk input handled, always
```

Sentinel `while` outside, `if/elif/else` dispatch inside, an `else` for nonsense. Learn this shape; you'll write it from memory within a month.

Two string helpers ride along: `.strip()` trims stray spaces and newlines off what `input()` returns; `.lower()` (next slide) lowercases it. String methods, the same dot-call shape as `file.readline()`.

§3 · CAFÉ take_order: many items, one loop

One customer, several items, unknown count → a sentinel `while` with a running total:

```
def take_order():
    order_total = 0.0
    item_count = 0
    still_ordering = True
    while still_ordering:
        product = input("  Product (or 'done'): ").strip().lower()
        if product == "done":
            still_ordering = False
        else:
            price = price_of(product)
            if price == 0.0:
                print("    Sorry, we don't sell that.")
            else:
                quantity = int(input("  Quantity: "))
                order_total = order_total + price * quantity
                item_count = item_count + quantity
                ...   # confirm the line
    to_pay = apply_discount(order_total)
    ...                   # the receipt: items, discount notice, total
    return to_pay
```

§3 · TRACE Run the loop by hand

Customer orders **2 latte**, then **1 muffin**, then **done**. Track the variables:

step	**input**	`product`	`price`	`quantity`	`order_total`
start					`0.00`
1	latte	latte	3.20	2	`6.40`
2	muffin	muffin	2.40	1	`8.80`
3	done	done	2.40	1	`8.80` → return

Tracing a loop line-by-line is an **exam skill**. Do it slowly; trust the table, not your gut. *(And `price_of`? An `if/elif` chain over our products: painful, and exactly the pain Week 4's dictionary erases.)*

Break

10–15 minutes

*When we come back: the **other** loop (`for`), what "iterable" really means, the day summary, and how professionals actually grow a program.*

Think about it: you drew the muffin rush before we coded it. Did drawing first change what you typed?

☎ *Answer on PollEv when we come back.*

§4 · CONCEPT for: one element at a time

Remember Week 2's argument vector? Print its contents **one by one**:

```
from sys import argv

print("Arguments to this program:", end=" ")
for arg in argv:
    print(" " + arg, end=" ")
print()
```

```
$ python3 argv_tour.py 11 22 33
Arguments to this program:  argv_tour.py  11  22  33
```

```
for variable in sequence:
    command        # once per element; variable takes each value in turn
```

`while` = *until a condition flips.* `for` = *for each element of a sequence.* Could you do this with `while`? Yes, and it's needlessly complicated. Match the loop to the job.

What counts as a sequence? Iterables

Strings are sequences of characters:

```
>>> for letter in "Hello, World!":
...     print(letter + "-", end=" ")
H- e- l- l- o- ,-  - W- o- r- l- d- !-

>>> for letter in "Hello, World!":
...     if letter == "a" or letter == "e" or letter == "i" \
...             or letter == "o" or letter == "u":
...         print(letter)
e
o
o
```

Loops and `if`s compose: that vowel filter is this week's two ideas working together.

The same word, two jobs: `in` as a test

You've been writing `for letter in ...` all afternoon. The keyword has a second job: **membership**, the yes/no test of "is this inside that?".

```
>>> "e" in "espresso"
True
>>> "decaf" in "double espresso, extra hot"
False
```

It answers a boolean, so it slots straight into an `if`. And yes: the vowel filter you just saw collapses to `if letter in "aeiou":` (the `==` chain showed how ifs compose; `in` is what you'll actually type). Next week the same test works on every collection we meet: lists, sets, dicts, unchanged.

Not everything iterates

```
>>> for n in 1234:
...     print(n)
```

```
TypeError: 'int' object is not iterable
```

An integer is **one value**, not a sequence of digits. But Week 1's conversions fix it:

```
>>> for n in str(1234):
...     print(n)
1
2
3
4
```

`str(1234)` *is* a sequence: the characters `"1"` `"2"` `"3"` `"4"`. Read your error messages: *"is not iterable"* tells you exactly what went wrong.

§4 · CONCEPT `range()`: sequences made to order

Four shapes, all examinable:

```
>>> for n in range(1, 5):      # m to n-1:        1 2 3 4
>>> for n in range(5):         # 0 to n-1:        0 1 2 3 4
>>> for n in range(0, 10, 3):  # with a step:     0 3 6 9
>>> for n in range(5, 0, -1):  # negative step:   5 4 3 2 1
```

- The end value is **never included:** `range(1, 5)` stops at 4.
- Default start `0`, default step `1`; a negative step counts down (mind the start/end!).
- `range(n)` + `for` = “do this n times”: Week 1’s receipt divider, the clean way.

§4 · CAFÉ Files are iterable: the day summary

A file is a **sequence of lines**, so `for` reads it naturally:

```
def day_summary():
    count = 0
    total = 0.0
    for line in open("receipts.txt"):
        line = line.strip()
        if line != "":
            count = count + 1
            total = total + float(line)
    print(f"  Orders today:  {count}")
    print(f"  Total revenue: {total:.2f}")
```

`for line in open(...)` is our course's one-pass read (when the loop ends, garbage collection closes the file for us; the books write `with open(...)`, which does the same job more formally). Each `line` arrives with its `\n` still attached; that's what `.strip()` removes. *(We cleared this file at open-up, so it exists; reading a maybe-missing file safely is Week 5's business.)*

Starred expressions

An iterable can be **unpacked** into separate print arguments with `*`:

```
>>> print(range(5))
range(0, 5)                          # the object, not the numbers
>>> print(*range(5))
0 1 2 3 4                            # unpacked!
>>> print(*"Hello, World!", sep="-")
H-e-l-l-o-,- -W-o-r-l-d-!
```

Handy for printing, occasionally on exams, and a taste of how flexible `print()` really is (`sep=` sets the separator between arguments, a cousin of Week 1's `end=`).

§4 · TRY IT Your turn (3 min)

1. Print a receipt divider, the line `----------` (ten dashes), using `for` and `range()`, without typing ten dashes. *(Hint: `print("-", end="")`, then one `print()`.)*
2. Loop over `"The Daily Grind"` and **count** the spaces. *(A counter + an `if`; you own both now.)*
3. Predict: what does `for n in range(10, 0, -2): print(n)` produce?

§5 · CAFÉ How professionals grow a program

Today's register wasn't written top-to-bottom. It **grew in versions**, and after *every* version, we ran it:

1. **v1, the broad idea:** menu printed once, `if/elif` dispatch, no loop. *Test: every choice, and watch junk like `"something"` fall through in silence; v4 fixes that.*
2. **v2, persistence:** `save_receipt`/`save_revenue` functions *(docstrings from day one). Test again.*
3. **v3, decisions:** the happy-hour discount. *Test the boundaries: 9.99, 10.00, 10.01.*
4. **v4, the menu loop:** wrap it in `while`, add the `else` for wrong choices. *Test: can junk input kill it now?*

Redraw the activity diagram before each version: bigger diagrams get a multi-way decision node  and *expand-activity* boxes  (worth recognizing; the exam asks for the four basic shapes). Diagram → build → **test** → **repeat.**

The first-morning crash (a preview of Week 5)

What happens the *very first* time the café opens and tries to read yesterday's revenue?

```
>>> revenue_file = open("revenue.txt")
FileNotFoundError: [Errno 2] No such file or directory: 'revenue.txt'
```

There *is* no yesterday. The program dies on line one. Our fix **today** is honest but crude, and it only protects the file we read mid-day: we create `receipts.txt` fresh at open-up (`open("receipts.txt", "w").close()`), so *that* one always exists. Reading yesterday's `revenue.txt` back is something the café does not do yet, and the week it starts is the week it needs the real fix (a program that survives missing files, bad numbers, and worse): **Week 5, errors and exceptions.** Remember this crash; we'll meet it again, armed.

Pause-per-line

A tiny trick from the file-reading world: display a file one line at a time, waiting for the reader:

```
def show_specials():
    """Display specials.txt, pausing after each line."""
    for line in open("specials.txt"):
        print(line, end="")
        dummy = input("  -- ENTER for more -- ")
```

`for`-over-a-file + an `input()` used purely as a pause. Three concepts from two weeks, composed into a feature. *(That's fluency: familiar tools, new combinations.)*

Three classic traps

```
# 1. = vs ==  (Week 2's bug, now inside a condition)
if choice = "1":        # SyntaxError -- assignment, not comparison

# 2. the boundary: > or >= ?
if total > 10.0:        # exactly 10.00 gets NO discount. Intended? Be consistent.

# 3. the infinite loop
while choice != "0":
    print("menu")       # forgot to read `choice` again -> loops forever (CTRL+C to escape)
```

Most beginner loop-bugs are one of these three. When a program hangs, check trap #3 first: *can the condition ever change?*

Where the café is now

Runnable snapshot: **`cafe_w3.py`:** greets you by name (`register_sayings` is still with us), serves customers **until close** through the menu loop, **decides** (the discount acts, cliffhanger paid), logs every receipt, and reads the day back with a `for`.

But notice the wall we just hit:

> *Our order is a single **running total**. We literally **cannot list** the items on the bill; a number can't remember what made it up. And `price_of()` is an `if/elif` chain that grows by two lines per product.*

Both pains have the same cure. **Next week: lists, sets, tuples, and the dictionary that eats that chain.**

Exam pointers

The exam can ask you to: *draw* an activity diagram for a described process (use the official symbols!); *hand-trace* a `while`/`for` (the table method); *find the error* (the three traps, the dangling-else); rewrite a `while` as a `for` (or back); state what `range(5, 0, -1)` produces.

Exercises & self-study

(optional; full map on Brightspace):

- **Self-check 3** → Brightspace tonight: about fifteen anonymous minutes, closes Thursday 23:59. We open next week on whatever the class lost.
- **Practical sheet** → Brightspace: Monday's practical works today's material through the week-3 sheet; solutions after the last session.
- **Project groups:** still ungrouped? The group form is on Brightspace; the deadline is the **end of week 5**, and the project launches on the Friday of week 6.
- **py4e** *(free, recommended)*: Ch 3 (conditionals) + Ch 5 (loops): videos + auto-graded exercises. *Heads-up: Ch 3 ex 2 and Ch 5 ex 1 use* `try`/`except`*; that's our Week 5; do the others now.*
- **ATBS** *(free alternative)*: Ch 2 (flow control: flowcharts, like us!) + Ch 3 (loops). Its two mini-games are **fully doable today**, on last week's `random` module. · **LPTHW** *(owners)*: Ex 29–36, 34 and 36 optional, plus Ex 37 as a symbol reference.

This week's checklist: tick before the practical

- ☐ I can plan before typing: pseudo-code → UML activity diagram → code, and translate in both directions.
- ☐ I can decide with `if`/`elif`/`else`, read nested ifs, and explain the dangling-else.
- ☐ I can validate input with a `while` loop (the muffin-rush pattern).
- ☐ I can run a menu loop (`while choice != "0":`) and say why one hangs.
- ☐ I can loop with `for` over argv, strings, files, use all four `range()` shapes, and test membership with `in`.
- ☐ I grow programs in small, tested versions: docstrings on, diagram first.

Anything unticked → that section of this deck, then the practical sheet.

Next week: give the order a memory with **lists, sets, tuples, dicts**, and the café reads its customers' minds (well, their reviews).